Apache Airflow | DAG Practice Handout

Two hands-on exercises using your existing Killercoda Docker setup. No new installation, external files, or solutions.

Before you begin

Use the existing airflow container and the Airflow UI on port 8080. Run the commands below in the Killercoda terminal:

```
docker ps
docker exec -it airflow bash
cd /opt/airflow/dags
```

Exercise 1 | Hotel Room Readiness Report

Objective: Build a four-task DAG using PythonOperator, parallel dependencies, and XCom. The workflow should prepare a housekeeping summary.

Input data (define this list inside the first Python function):

```
rooms = [
    {"room": 101, "status": "Ready",    "cleaning_min": 0},
    {"room": 102, "status": "Cleaning", "cleaning_min": 25},
    {"room": 103, "status": "Ready",    "cleaning_min": 0},
    {"room": 104, "status": "Cleaning", "cleaning_min": 40},
    {"room": 105, "status": "Cleaning", "cleaning_min": 30}
]
```

Tasks to implement

1. load_rooms: Return the list of rooms from the function.
2. find_rooms_to_clean: Use XCom to read the list and return room numbers whose status is Cleaning.
3. calculate_cleaning_minutes: Use XCom to calculate the total cleaning minutes for those rooms.
4. generate_housekeeping_report: Read both results from XCom and print a summary.

Required dependency structure

```
load_rooms >> [find_rooms_to_clean, calculate_cleaning_minutes]
[find_rooms_to_clean, calculate_cleaning_minutes] >> generate_housekeeping_report
```

Execution and verification

Create hotel_readiness_dag.py using cat > hotel_readiness_dag.py <<EOF; paste your DAG code and end with EOF on its own line. Use schedule=None and catchup=False. Validate with:

```
python -m py_compile hotel_readiness_dag.py
airflow dags list
```

Trigger the DAG in the Airflow UI. Verify that the two calculation tasks run independently and that the final report prints the rooms needing cleaning and total cleaning time.

Exercise 2 | IT Incident Priority Routing

Use the same Killercoda container and Airflow UI. This exercise adds branching, retries, and trigger rules.

Objective

Build a DAG that routes an IT incident to the appropriate response task based on severity. Simulate a temporary failure on the urgent path and verify Airflow retry behavior.

Input data (define inside the first Python function)

```
incident = {
    "ticket_id": "INC-402",
    "service": "Email",
    "severity": "High",
    "reported_by": "Operations"
}
```

Tasks to implement

1. load_incident: Return the incident dictionary.
2. choose_response: Use BranchPythonOperator and XCom to route High/Critical severity to urgent_response; all other severities to standard_response.
3. urgent_response: Print an escalation message. Simulate a failure on its first attempt, then succeed on retry. Configure retries=1 and a short retry_delay.
4. standard_response: Print a normal service-desk assignment message.
5. write_audit: Print the incident ID and completion message. Configure a trigger rule that permits this task to run when one branch succeeds and the other is skipped.

Required dependency structure

```
load_incident >> choose_response
choose_response >> [urgent_response, standard_response]
[urgent_response, standard_response] >> write_audit
```

Execution and verification

Create it_incident_routing_dag.py using the same cat > filename.py <<EOF approach. Use the same PythonOperator import style as your earlier Airflow exercises:

```
from airflow.providers.standard.operators.python import (
    PythonOperator, BranchPythonOperator
)
```

Validate and trigger

```
python -m py_compile it_incident_routing_dag.py
airflow dags list
```

1. Run with severity High. Check the urgent task retries and the standard task is Skipped.
2. Change severity to Low and trigger again. Check that the standard task succeeds.
3. Verify that write_audit executes after the selected branch succeeds.
4. Inspect the Graph view and task logs for both DAG runs.

Submission

Submit the two DAG Python files and screenshots of each successful Graph view and final task log. Do not submit only the Python code: both DAGs must run in Airflow.